C PROGRAMMING • DATA STRUCTURES FOUNDATIONS

STRUCTURES

A Complete Exam-Oriented Study Guide

University Exams | Competitive Exams | Teacher Recruitment Tests

INTRODUCTION

Definition of Structure

A **structure** is a user-defined data type in C that allows the programmer to combine data items of **different data types** under a single name. Each data item inside a structure is called a **member** or **field**. Structures let a programmer create a single logical record (for example, a student, an employee, or a book) that groups related but dissimilar pieces of information together.

> **DEFINITION**
> A structure is a collection of variables, possibly of different data types, stored in contiguous memory and grouped together under a single name, so that the group can be treated as a single unit while each member can still be accessed individually.

A Structure in Action

C CODE

```c
#include <stdio.h>

struct Student {          /* structure definition: the template */
    int roll;
    char name[20];
    float marks;
};

int main() {
    struct Student s1 = {1, "Meena", 87.5};   /* structure variable */

    printf("Roll: %d\n", s1.roll);
    printf("Name: %s\n", s1.name);
    printf("Marks: %.1f\n", s1.marks);
    return 0;
}
```

OUTPUT

```
Roll: 1
Name: Meena
Marks: 87.5
```

Notice how `roll` (int), `name` (char array), and `marks` (float) — three different data types — are stored together under the single name `Student`, exactly matching the definition above.

Features of Structures

- Groups logically related but dissimilar data items into a single unit.
- Members can be of any valid data type, including arrays, pointers, and even other structures.
- Memory for all members is allocated together in a contiguous block (subject to padding/alignment rules).
- A structure itself becomes a new user-defined data type once declared using the `struct` keyword.
- Multiple variables can be created from a single structure template.

Advantages of Structures

- **Logical grouping:** Related data (e.g., name, roll number, marks of a student) is stored together, improving readability.
- **Single unit handling:** A structure variable can be passed to functions, assigned, or copied as a single unit.
- **Reusability:** Once a structure type is defined, it can be reused to declare any number of variables.
- **Foundation for advanced data structures:** Structures are the building blocks for linked lists, trees, graphs, and stacks/queues implemented using linked allocation.
- **Better program organization:** Reduces the need for a large number of unrelated, separately named variables.

Structure vs Array — Quick Comparison

Basis	Array	Structure
Data type	Homogeneous (same type)	Heterogeneous (different types allowed)
Declaration keyword	None special — uses base type	`struct` keyword
Element access	By index, e.g. `arr[2]`	By member name, e.g. `s.roll`
Memory	Contiguous, fixed-size same-type blocks	Contiguous, but member sizes may differ (with padding)
Use case	Same-type list of values	Single record with multiple attributes

Diagram: Structure Memory Organization

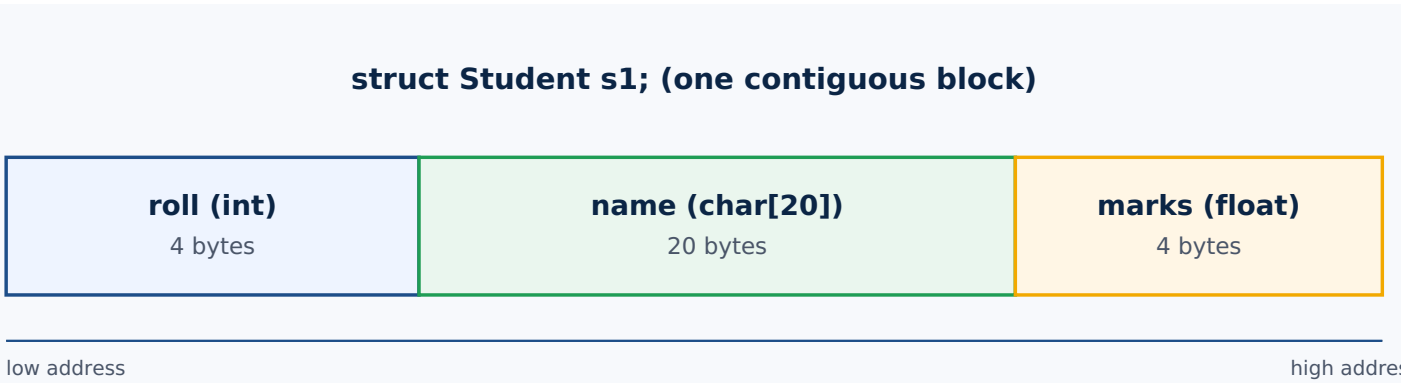


Fig 0.1 — Memory layout of a structure holding an int, a float, and a char array (illustrative; actual byte offsets depend on compiler alignment/padding rules).

Illustrative Declaration

C CODE

```c
struct Student {
    int roll;
    char name[20];
    float marks;
};
```

Here `roll` (int), `name` (char array) and `marks` (float) are three different data types combined into one heterogeneous unit named `Student`. Compare this with an array `int marks[50];` where every element must be an int — a homogeneous collection.

CHAPTER 1

Structure as a Homogeneous and Heterogeneous Collection

1.1 What Makes a Collection Homogeneous or Heterogeneous

To understand why structures are important, it is necessary to understand the difference between homogeneous and heterogeneous collections of data. The distinction comes down to one question: **can the elements of the collection be of different data types, or must they all be the same?** This single question determines whether a collection should be built as an array or as a structure.

1.2 Homogeneous Collection: The Array

A collection in which **all elements are of the same data type** is called a homogeneous collection. In C, the **array** is the classic example of a homogeneous collection — every element of an array must be of the same type (all int, all float, all char, etc.). Because every element occupies exactly the same number of bytes, the compiler can locate any element instantly using a single formula: `base_address + (index × element_size)`. This is precisely why array indexing is so fast and why arrays cannot mix data types — the whole efficiency of index-based access depends on every element being identical in size and type.

1.3 Heterogeneous Collection: The Structure

A collection in which **elements can be of different data types** is called a heterogeneous collection. The **structure** is the classic example of a heterogeneous collection in C — a single structure can hold an int, a float, and a character array together as one unit. Since members can differ in size and type, they cannot be located using a simple index formula; instead, the compiler calculates a fixed **offset** for each member at compile time, based on the order in which members are declared and the alignment rules of the target platform. This is why structure members are accessed by name (using the dot or arrow operator) rather than by numeric index.

1.4 Side-by-Side Example

Array (Homogeneous) vs Structure (Heterogeneous) in One Program

C CODE

```c
#include <stdio.h>

struct Student {          /* heterogeneous: int + char array + float */
    int roll;
    char name[20];
    float marks;
};

int main() {
    int marksArray[3] = {70, 85, 90};        /* homogeneous: all int */
    struct Student s1 = {1, "Aman", 78.5};    /* heterogeneous record */

    printf("Array elements (all int): %d %d %d\n",
           marksArray[0], marksArray[1], marksArray[2]);

    printf("Structure members (mixed types): %d, %s, %.1f\n",
           s1.roll, s1.name, s1.marks);
    return 0;
}
```

OUTPUT

```
Array elements (all int): 70 85 90
Structure members (mixed types): 1, Aman, 78.5
```

The array `marksArray` can only ever store `int` values, indexed as `marksArray[0]`, `marksArray[1]`, etc. The structure `s1` stores three different types together and each is accessed by its own name — `s1.roll`, `s1.name`, `s1.marks` — never by a numeric index.

1.5 Detailed Comparison Table

Aspect	Homogeneous Collection (Array)	Heterogeneous Collection (Structure)
Data type of elements	Same type throughout	Can differ from member to member
Real-world analogy	A row of identical lockers	A student's ID card (name, roll no., DOB, marks)
Access method	Index / subscript	Member name using dot (.) operator
Address calculation	base + (index × element size)	Fixed per-member offset, resolved at compile time
Typical use	List of marks, list of temperatures	A single record such as a student or employee
Can new types be mixed in?	No — violates homogeneity	Yes — that is the entire purpose of a structure

1.6 Diagram: Homogeneous vs Heterogeneous Layout

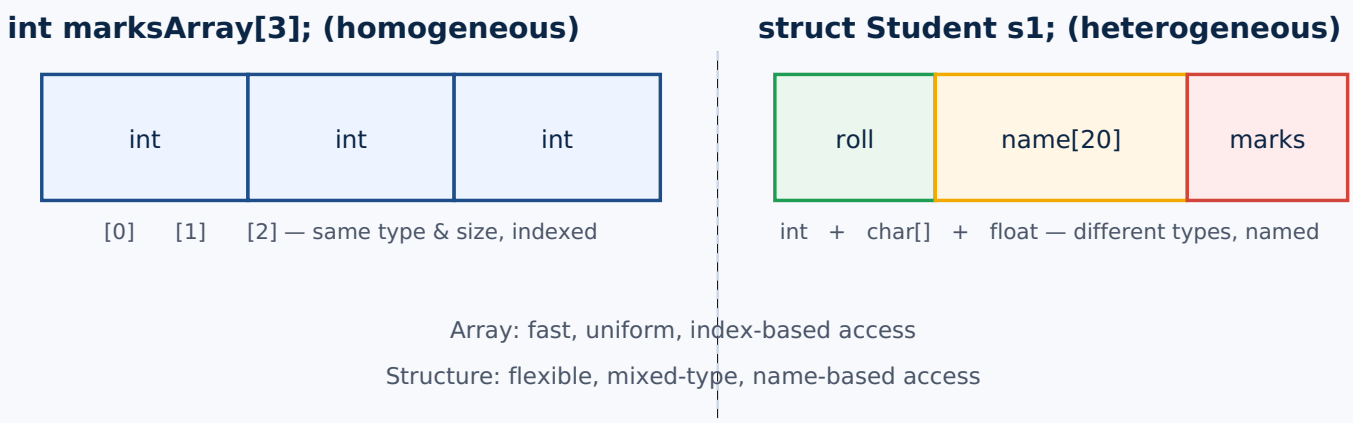


Fig 1.1 — An array stores identical, indexable slots; a structure stores differently-typed, named members side by side.

1.7 Common Misconceptions

COMMON MISTAKE

Assuming an array can be made "heterogeneous" simply by using a generic or `void*` type for its elements. While this is technically possible in advanced C, it defeats the purpose of an array's fast, fixed-size indexing and is not what is meant by a "heterogeneous collection" in this context — the structure remains the correct, standard tool for grouping different data types.

EXAM TIP

A very common exam question is: "Why is a structure called a heterogeneous data type while an array is homogeneous?" Answer using the definitions above along with one example each, and mention the underlying reason — how each collection's memory is addressed (index formula vs compile-time offsets).

Chapter 1 Summary

- A structure is a heterogeneous, user-defined collection of related data items (members) under one name.
- Arrays are homogeneous; structures are heterogeneous — this is the single most important distinguishing fact.
- Arrays use fast index-based addressing because every element is identical in size; structures use compile-time member offsets because members can differ in type and size.
- Both are essential tools: arrays for uniform lists, structures for multi-attribute records.

CHAPTER 2

Possible Applications of Structures

2.1 Why Structures Are Needed in Real Applications

Almost every real-world entity has several attributes of different data types. A structure allows a programmer to model such an entity naturally as a single record, instead of maintaining many separate, unrelated variables or arrays.

2.2 Student Record Management

A student record typically needs a roll number (int), a name (char array), and marks/percentage (float). A structure groups these into one `Student` record, and an array of such structures can represent an entire class.

C CODE

```c
struct Student {
    int roll;
    char name[30];
    float percentage;
};
```

2.3 Employee Database

An employee record needs an employee ID, name, department, and salary — again a mix of int, char array, and float/double. Structures are used to build employee database systems, payroll systems, and HR management tools.

C CODE

```c
struct Employee {
    int empId;
    char name[30];
    char department[20];
    double salary;
};
```

2.4 Library Management

A library management system stores book records with an accession number, title, author, and availability status — a natural fit for a structure that can be stored in arrays or linked lists for the full catalogue.

C CODE

```c
struct Book {
    int accessionNo;
    char title[50];
    char author[30];
    int isIssued;   /* 0 = available, 1 = issued */
};
```

2.5 Banking Records

Bank account records combine an account number, customer name, account type, and balance. Structures allow banking software to store, update, and process each account as a single logical unit.

C CODE

```c
struct Account {
    long accNo;
    char holderName[30];
    char accType[10];
    float balance;
};
```

2.6 Other Practical Examples

- **Inventory / stock management:** item code, name, quantity, price.
- **Hospital patient records:** patient ID, name, age, diagnosis, ward number.
- **Graphics programming:** a `Point` structure with x and y coordinates, or a `Color` structure with red, green, blue components.
- **Date/time handling:** a `Date` structure with day, month, and year members.
- **Complex numbers:** a structure with real and imaginary parts for mathematical computation.

EXAM TIP

When asked to "list applications of structures," always support each application with the specific members it would contain — examiners award marks for concrete examples, not just names of domains.

Chapter 2 Summary

- Structures model real-world entities: students, employees, books, bank accounts, patients, and more.
- Each application combines attributes of different data types into one coherent record.
- Arrays of structures (Chapter 7) are typically used to represent entire collections such as a class of students or a full library catalogue.

CHAPTER 3

Syntax of Declaring Structure

3.1 General Syntax of Structure Declaration

SYNTAX

```
struct <tag_name> {
    data_type member1;
    data_type member2;
    ...
    data_type memberN;
};   /* semicolon is compulsory */
```

Here `struct` is a keyword, `tag_name` is the name given to this new structure type, and the members listed inside the braces are the individual data items. The closing brace **must** be followed by a semicolon — a very common syntax error is forgetting this semicolon.

3.2 Defining a Structure Type

Declaring a structure only creates a **template** or **blueprint**; no memory is allocated at this point. Memory is allocated only when a structure **variable** is created from this template.

1 Declaring a Structure Type

C CODE

```
struct Student {
    int roll;
    char name[20];
    float marks;
};
```

This declares a new type called `struct Student`. No variable of this type exists yet, and no memory has been reserved.

3.3 Creating Structure Variables

There are two common ways to create (declare) variables of a structure type:

Method 1: Declare Variables Along with the Structure

SYNTAX

```
struct tag_name {
    data_type member1;
    data_type member2;
} var1, var2, ...;
```

Method 2: Declare Variables Separately, After the Structure

SYNTAX

```
struct tag_name var1, var2, ...;
```

2 Both Methods Illustrated

C CODE

```
/* Method 1: variables declared with structure */
struct Student {
    int roll;
    char name[20];
    float marks;
} s1, s2;

/* Method 2: variables declared later */
struct Student s3, s4;
```

3.4 Using typedef with Structures

The `typedef` keyword can be used to create an alias for the structure type, removing the need to repeat the `struct` keyword every time a variable is declared.

C CODE

```c
typedef struct {
    int roll;
    char name[20];
    float marks;
} Student;

Student s1, s2;   /* no need to write "struct" */
```

3.5 Rules for Structure Declaration

- The structure declaration must end with a semicolon.
- Member names inside a structure must be unique within that structure.
- A structure declaration only creates a type; it does not allocate memory unless a variable is created.
- Members cannot be initialized with values at the point of the type declaration in traditional C (only at variable creation, see Chapter 4).
- The tag name is optional if variables are declared immediately after the closing brace, but omitting it prevents creating further variables later (unless `typedef` is used).

COMMON MISTAKE

Forgetting the semicolon after the closing brace } of the structure declaration — this causes a compilation error, often reported on the next line of code, confusing beginners.

Chapter 3 Summary

- Syntax: `struct tag_name { members... };`
- Declaring a structure only defines a template; memory is allocated only for variables.
- Variables can be declared with the structure, separately later, or via `typedef`.
- The trailing semicolon after the closing brace is compulsory.

CHAPTER 4

Initializing Structures

4.1 What Is Structure Initialization?

Initialization means assigning initial values to the members of a structure variable at the time it is declared, rather than assigning them one by one afterwards using the member operator.

4.2 Full Initialization Syntax

SYNTAX

```
struct tag_name var = {value1, value2, ..., valueN};
```

1 Full Initialization

C CODE

```c
#include <stdio.h>
struct Student {
    int roll;
    char name[20];
    float marks;
};

int main() {
    struct Student s1 = {101, "Aman", 89.5};
    printf("Roll: %d\nName: %s\nMarks: %.1f", s1.roll, s1.name, s1.marks);
    return 0;
}
```

OUTPUT

```
Roll: 101
Name: Aman
Marks: 89.5
```

4.3 Partial Initialization

If fewer values are provided than the number of members, the remaining members are automatically initialized to zero (for numeric types) or an empty string / null characters (for char arrays). Members cannot be skipped from the middle in old-style positional initialization — only trailing members can be left out.

2 Partial Initialization

C CODE

```c
struct Student s2 = {102, "Riya"};   /* marks not given */
printf("Roll: %d, Name: %s, Marks: %.1f", s2.roll, s2.name, s2.marks);
```

OUTPUT

```
Roll: 102, Name: Riya, Marks: 0.0
```

The uninitialized `marks` member is automatically set to 0 by the compiler.

4.4 Designated Initializers (C99 and later)

Modern C (C99 onwards) allows initializing specific members by name, in any order, using designated initializers.

C CODE

```c
struct Student s3 = {.roll = 103, .marks = 76.0, .name = "Karan"};
```

4.5 Nested Initialization

When a structure contains another structure as a member (see Chapter 11), initialization uses nested braces for the inner structure's values.

3 Nested Structure Initialization

C CODE

```c
struct Date {
    int day, month, year;
};
struct Student {
    int roll;
    char name[20];
    struct Date dob;   /* nested structure member */
};

struct Student s4 = {104, "Neha", {12, 8, 2004}};
printf("%d-%d-%d", s4.dob.day, s4.dob.month, s4.dob.year);
```

OUTPUT

```
12-8-2004
```

4.6 Initializing After Declaration (Member-wise)

A structure variable can also be initialized member-by-member after declaration, using the member operator (fully explained in Chapter 6):

C CODE

```c
struct Student s5;
s5.roll = 105;
strcpy(s5.name, "Amit");
s5.marks = 91.2;
```

Note: string members must be assigned using `strcpy()`, not the = operator, because C arrays cannot be assigned directly.

COMMON MISTAKE

Trying to assign a string directly to a char-array member using `s5.name = "Amit";` — this is illegal in C and causes a compile-time error. Always use `strcpy()`.

EXAM TIP

Remember: whole-structure assignment (`s2 = s1;`) IS allowed in C and copies all members at once, but direct string assignment to a char-array member is NOT allowed.

Chapter 4 Summary

- Full initialization: `struct tag var = {v1, v2, ...};`
- Partial initialization sets remaining members to zero/empty.
- Designated initializers (C99+) allow named, out-of-order initialization.
- Nested structures are initialized using nested curly braces.
- String members require `strcpy()`, never direct assignment.

CHAPTER 5

Structure Variables

5.1 What Is a Structure Variable?

A structure **variable** is an actual instance of a structure type for which memory is allocated. While the structure declaration is only a blueprint, a structure variable is a concrete object in memory that holds real values for each member.

5.2 Declaring and Using Structure Variables

SYNTAX

```
struct tag_name variable_name;
```

1 Declaring and Using a Structure Variable

C CODE

```c
#include <stdio.h>
struct Student {
    int roll;
    char name[20];
    float marks;
};

int main() {
    struct Student s1;          /* structure variable declared */
    s1.roll = 1;
    strcpy(s1.name, "Ravi");
    s1.marks = 88.0;

    printf("%d %s %.1f", s1.roll, s1.name, s1.marks);
    return 0;
}
```

OUTPUT

```
1 Ravi 88.0
```

5.3 Memory Allocation for Structure Variables

When a structure variable is declared, the compiler allocates a contiguous block of memory large enough to hold all its members. The total size is usually the sum of the sizes of individual members, but may be slightly larger due to **padding** added by the compiler for memory alignment purposes.

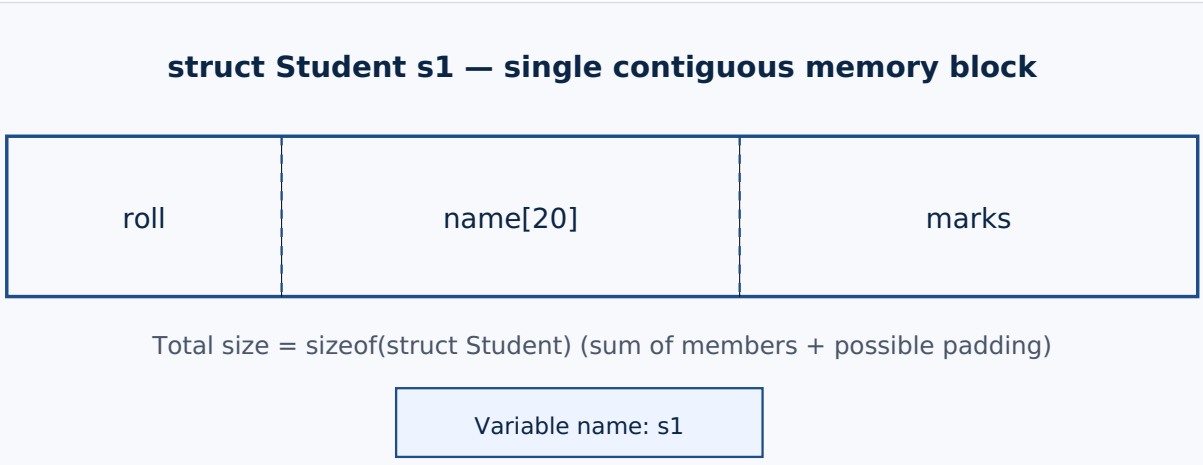


Fig 5.1 — A structure variable s1 of type `struct Student` occupies one contiguous memory block containing all its members.

2 Size of a Structure Variable

C CODE

```c
printf("Size = %lu bytes", sizeof(s1));
```

OUTPUT

```
Size = 28 bytes   /* value varies by compiler/alignment */
```

The exact size depends on the compiler, the target architecture, and padding/alignment rules — it is not simply the arithmetic sum of member sizes.

5.4 Multiple Structure Variables

Any number of variables can be created from the same structure type, and each variable gets its own independent copy of the members in memory.

C CODE

```
struct Student s1, s2, s3;   /* three independent instances */
```

5.5 Copying Structure Variables

Unlike arrays, C allows direct assignment of one structure variable to another of the **same type**; all members are copied automatically (a member-wise shallow copy).

C CODE

```
struct Student s2 = s1;   /* copies roll, name, and marks from s1 into s2 */
```

EXAM TIP

"Can we assign one structure variable to another directly?" — Yes, if they are of the same structure type. This is different from arrays, which cannot be assigned using = directly.

Chapter 5 Summary

- A structure variable is a real instance of a structure type with allocated memory.
- Memory is contiguous but may include padding for alignment; use `sizeof()` to find actual size.
- Multiple independent variables can be created from one structure type.
- Whole structure variables of the same type can be assigned directly using =.

CHAPTER 6

Accessing Structure Elements Using Member Operator (.)

6.1 The Member Access (Dot) Operator

Individual members of a structure variable are accessed using the **dot operator** (`.`), also called the **member access operator** or **period operator**. It connects the structure variable name with the member name.

SYNTAX

```
structure_variable.member_name
```

6.2 Reading Structure Members

Once a structure variable holds values, its members can be read at any time by writing the variable name, followed by a dot, followed by the member name. The result behaves exactly like an ordinary variable of that member's type, so it can be printed, used in an expression, or copied into another variable.

1 Reading Members with the Dot Operator

C CODE

```c
#include <stdio.h>
struct Student {
    int roll;
    char name[20];
    float marks;
};

int main() {
    struct Student s1 = {201, "Sonal", 78.5};
    printf("Roll No: %d\n", s1.roll);
    printf("Name: %s\n", s1.name);
    printf("Marks: %.1f\n", s1.marks);
    return 0;
}
```

OUTPUT

```
Roll No: 201
Name: Sonal
Marks: 78.5
```

6.3 Modifying Structure Members

Since the dot operator provides direct access to a member's location in memory, it can be used on the left-hand side of an assignment to change that member's value at any point after the structure variable has been declared. Numeric members can be updated with a simple = assignment, while character-array (string) members must be updated using `strcpy()`.

2 Modifying Members with the Dot Operator

C CODE

```c
#include <stdio.h>
#include <string.h>

struct Student {
    int roll;
    char name[20];
    float marks;
};

int main() {
    struct Student s1 = {201, "Sonal", 78.5};

    s1.marks = 82.0;              /* modify a numeric member  */
    strcpy(s1.name, "Sonal K.");  /* modify a char array member */

    printf("Updated Marks: %.1f\n", s1.marks);
    printf("Updated Name: %s\n", s1.name);
    return 0;
}
```

OUTPUT

```
Updated Marks: 82.0
Updated Name: Sonal K.
```

6.4 Diagram: Member Access Using the Dot Operator

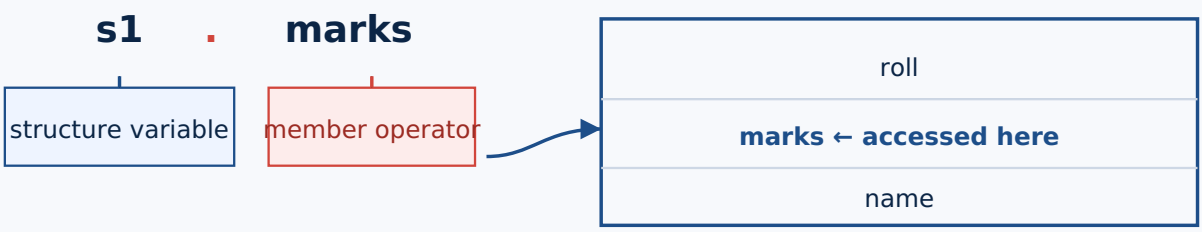


Fig 6.1 — The dot operator connects the structure variable to a specific member's location in memory.

6.5 Reading Input into Structure Members

Structure members can also be filled directly from user input using `scanf()`, by passing the address of each member (except array members, which already decay to an address) exactly as you would for an ordinary variable.

3 Taking Input Using scanf()

C CODE

```c
#include <stdio.h>

struct Student {
    int roll;
    char name[20];
    float marks;
};

int main() {
    struct Student s2;

    printf("Enter roll, name, marks: ");
    scanf("%d %s %f", &s2.roll, s2.name, &s2.marks);

    printf("You entered -> Roll: %d, Name: %s, Marks: %.1f\n",
           s2.roll, s2.name, s2.marks);
    return 0;
}
```

SAMPLE RUN

```
Enter roll, name, marks: 5 Priya 91.5
You entered -> Roll: 5, Name: Priya, Marks: 91.5
```

Note the & before `s2.roll` and `s2.marks` (address needed for scanf), but no & before `s2.name` since an array name already decays to an address.

6.6 Rules of the Member Operator

- The dot operator can only be used with a structure **variable** (or another structure directly), never with a structure pointer (pointers use `->`, see Chapter 15).
- Nested members are accessed by chaining dot operators, e.g. `s1.dob.year`.
- The member operator has very high precedence, evaluated before most other operators.

COMMON MISTAKE

Using the dot operator on a structure **pointer**, e.g. `ptr.roll` where `ptr` is a pointer — this is a compile-time error. Use `ptr->roll` or `(*ptr).roll` instead.

Chapter 6 Summary

- The dot operator (`.`) accesses members of a structure variable: `var.member`.

- It can be used both to read and to modify member values.
- String members need `strcpy()` for modification, not direct assignment.
- Nested members are accessed by chaining dots: `var.inner.member`.

CHAPTER 7

Arrays of Structures

7.1 Concept

An **array of structures** is a collection of structure variables of the same type, stored contiguously in memory and accessed using an index, just like an ordinary array. It combines the homogeneous grouping ability of arrays with the heterogeneous member capability of structures — making it ideal for storing multiple records of the same kind (e.g., an entire class of students).

7.2 Declaration and Initialization

An array of structures is declared the same way as an ordinary array, using the structure's tag name as the base type. Each element of the array is one complete structure, so initializing the array means providing one set of curly-brace values per element, in order.

SYNTAX

```
struct tag_name array_name[size];

/* with initialization */
struct tag_name array_name[size] = {
    {value1, value2, ...},
    {value1, value2, ...},
    ...
};
```

1 Array of Structures — Declaration and Initialization

C CODE

```c
struct Student {
    int roll;
    char name[20];
    float marks;
};

struct Student class[3] = {
    {1, "Aman", 78.0},
    {2, "Riya", 85.5},
    {3, "Karan", 66.0}
};
```

7.3 Input and Output Operations

Because each element of the array is a full structure, reading or displaying an entire array of structures is most naturally done with a loop — the loop counter selects the element (`class[i]`), and the dot operator then reaches the individual members within it.

2 Reading and Displaying an Array of Structures

C CODE

```c
#include <stdio.h>
struct Student {
    int roll;
    char name[20];
    float marks;
};

int main() {
    struct Student class[3];
    int i;

    for (i = 0; i < 3; i++) {
        printf("Enter roll, name, marks for student %d: ", i + 1);
        scanf("%d %s %f", &class[i].roll, class[i].name, &class[i].marks);
    }

    printf("\n--- Student Records ---\n");
    for (i = 0; i < 3; i++) {
        printf("%d\t%s\t%.1f\n", class[i].roll, class[i].name, class[i].marks);
    }
    return 0;
}
```

SAMPLE OUTPUT

```
--- Student Records ---
1   Aman    78.0
2   Riya    85.5
3   Karan   66.0
```

7.4 Memory Layout of an Array of Structures

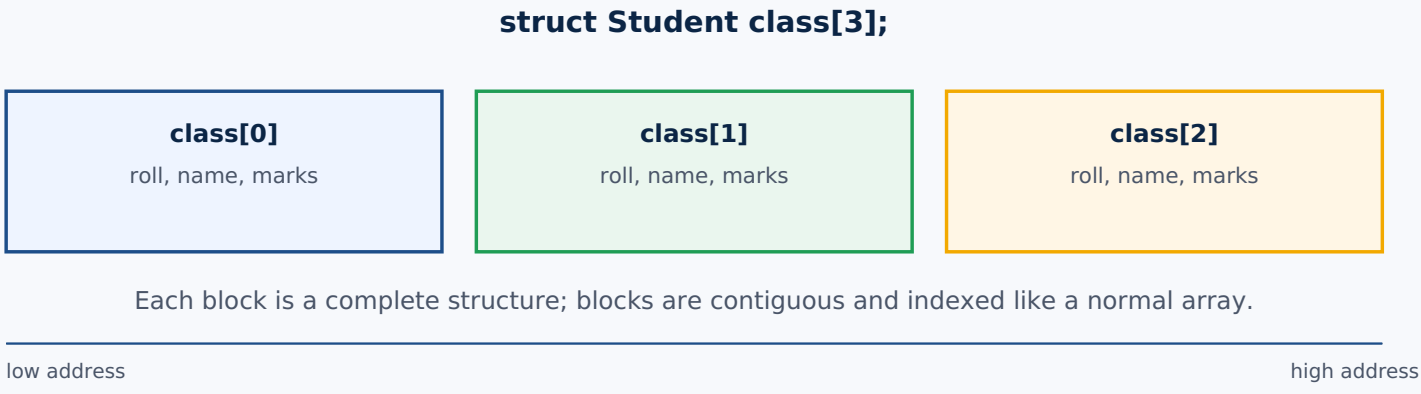


Fig 7.1 — Each element `class[i]` is itself a complete structure block; blocks are laid out contiguously one after another.

7.5 Practical Applications

- Storing an entire class of student records.
- Maintaining an employee database with a fixed number of employees.
- Storing multiple book entries in a small library catalogue.
- Building simple in-memory tables before moving to file-based or database storage.

EXAM TIP

An array of structures is often the exact answer expected when a question asks "how would you store records of multiple students/employees?" — pair it with a `for` loop for input/output.

Chapter 7 Summary

- An array of structures stores multiple records of the same structure type, indexed like a normal array.
- Declared as `struct tag_name arr[size];` and accessed as `arr[i].member`.
- Memory layout is a sequence of contiguous structure blocks.
- Ideal for representing collections such as classes, employee lists, or catalogues.

CHAPTER 8

Array as Member of Structure

8.1 Concept

Just as a structure can be one element of an array (Chapter 7), an **array can also be a member inside a structure**. This is extremely common — for example, a `char` array is almost always used to store a name or string inside a structure, and a numeric array can be used to store multiple marks/scores for a single student.

8.2 Declaring Array Members

SYNTAX

```
struct tag_name {
    data_type array_member[size];
    ...
};
```

1 Structure with an Array Member (Multiple Subject Marks)

C CODE

```c
#include <stdio.h>
struct Student {
    int roll;
    char name[20];
    float marks[5];   /* array member: marks in 5 subjects */
};

int main() {
    struct Student s1 = {1, "Amit", {80, 75, 90, 60, 85}};
    int i;
    float total = 0;

    for (i = 0; i < 5; i++) {
        printf("Subject %d: %.1f\n", i + 1, s1.marks[i]);
        total += s1.marks[i];
    }
    printf("Average = %.2f", total / 5);
    return 0;
}
```

OUTPUT

```
Subject 1: 80.0
Subject 2: 75.0
Subject 3: 90.0
Subject 4: 60.0
Subject 5: 85.0
Average = 78.00
```

8.3 Accessing Array Members

An array member is accessed by first using the dot operator to reach the member, then the usual subscript operator to reach a specific element: `structure_variable.array_member[index]`.

8.4 Applications

- Storing marks of a student in multiple subjects.
- Storing monthly sales figures for an employee/salesperson within one record.
- Storing a fixed-size history (e.g., last 7 days' temperature readings) inside a single structure.
- Representing multi-component data such as a matrix row or a set of coordinates.

EXAM TIP

Do not confuse "array of structures" (Chapter 7, many records) with "array as a member of structure" (Chapter 8, one record containing many values for one field). Questions often test this exact distinction.

Chapter 8 Summary

- An array can be declared as a member inside a structure, just like any other data type.

- Useful for storing multiple related values (marks, sales, readings) for a single record.
- Accessed as `var.arrayMember[index]`.
- Distinct from "array of structures" — the direction of nesting is reversed.

CHAPTER 9

Conceptual Difference Between Array and Structure Collection

9.1 Array vs Structure — The Core Idea

Although both arrays and structures are "collections" of data, they differ fundamentally in the **type uniformity** of their elements and in how those elements are accessed. This chapter consolidates and expands the comparison introduced in Chapter 1.

9.2 Homogeneous vs Heterogeneous Data (Revisited)

An array can only store elements of one single data type — this uniformity allows the compiler to compute the address of any element using a simple formula: `base_address + (index × size_of_element)`. A structure, in contrast, stores elements of possibly different types and sizes, so each member has its own offset from the start of the structure, computed at compile time based on member order and alignment.

9.3 Comprehensive Comparison Table

Basis of Comparison	Array	Structure
Nature of collection	Homogeneous	Heterogeneous
Keyword required	None (uses base type)	`struct`
Element/member access	Index using `[ ]`	Name using `.` or `->`
Memory address calculation	Simple arithmetic (base + i×size)	Fixed offsets determined at compile time
Assignment of whole collection	Not allowed directly (`arr2 = arr1;` is illegal)	Allowed directly for same type (`s2 = s1;`)
Size	`sizeof(type) × n`	Sum of member sizes + possible padding
Typical use	Storing a list of similar values	Storing a single multi-attribute record
Nesting	Arrays of arrays (multi-dimensional)	Structures within structures (nested), or arrays within structures
Passing to functions	Always passed by reference (decays to pointer)	Can be passed by value or by reference (pointer)

9.4 Advantages and Limitations

Array — Advantages

- Simple, fast, index-based access.
- Efficient for storing large sets of same-type data.

Array — Limitations

- Cannot mix data types.
- Cannot be assigned as a whole using =.

Structure — Advantages

- Groups dissimilar but related data logically.
- Can be assigned, passed, and returned as a single unit.

Structure — Limitations

- Slightly more memory overhead due to padding.
- Member access is name-based, requiring the tag/type to be known.

9.5 Array vs Structure — Comparison Chart

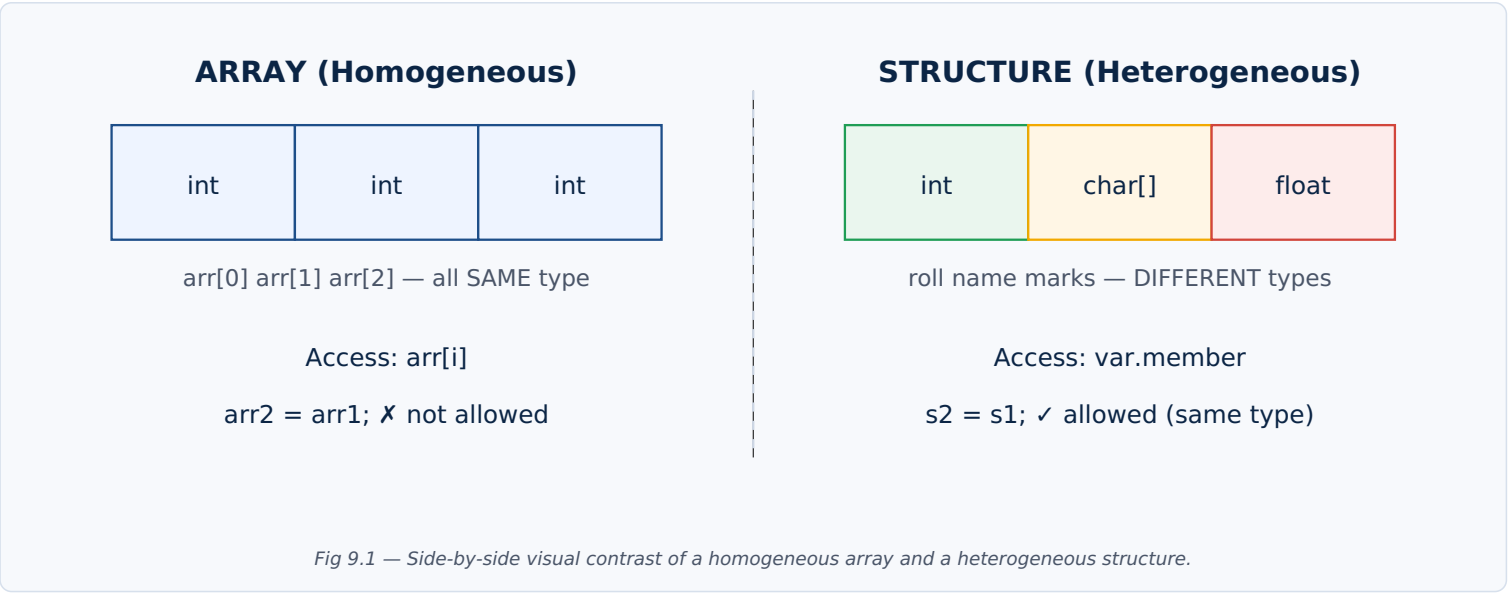


Fig 9.1 — Side-by-side visual contrast of a homogeneous array and a heterogeneous structure.

EXAM TIP

A favourite short-answer question: "Differentiate between array and structure (any four points)." Use rows 1, 3, 5, and 6 from the table above as a safe, high-scoring answer.

Chapter 9 Summary

- Arrays are homogeneous and index-accessed; structures are heterogeneous and name-accessed.
- Whole-array assignment is disallowed; whole-structure assignment (same type) is allowed.
- Both can be nested with each other: arrays of structures and arrays inside structures.

CHAPTER 10

Functions and Structures

10.1 Overview

Structures can interact with functions in three main ways: passing individual members as arguments, passing the entire structure (by value or by reference), and returning a structure from a function. This chapter focuses on the general working mechanism; Chapter 17 covers passing/returning structures in more formal exam-style detail.

10.2 Passing Structures to Functions (By Value)

When a structure is passed by value, a **complete copy** of the structure is made and given to the function. Any changes made inside the function to the copy do **not** affect the original structure in the caller.

1 Passing a Structure by Value

C CODE

```c
#include <stdio.h>
struct Point {
    int x, y;
};

void display(struct Point p) {   /* receives a COPY */
    printf("x = %d, y = %d\n", p.x, p.y);
    p.x = 100;   /* modifies only the local copy */
}

int main() {
    struct Point p1 = {5, 10};
    display(p1);
    printf("Original still: x = %d, y = %d", p1.x, p1.y);
    return 0;
}
```

OUTPUT

```
x = 5, y = 10
Original still: x = 5, y = 10
```

10.3 Passing Structures by Reference (Pointer)

To allow a function to modify the caller's original structure, the **address** of the structure is passed using a pointer. The function then uses the member selector operator -> (Chapter 15) to access members.

2 Passing a Structure by Reference

C CODE

```c
void update(struct Point *p) {
    p->x = 100;   /* modifies the ORIGINAL structure */
}

int main() {
    struct Point p1 = {5, 10};
    update(&p1);
    printf("x = %d, y = %d", p1.x, p1.y);
    return 0;
}
```

OUTPUT

```
x = 100, y = 10
```

10.4 Returning Structures from Functions

A function can also return a complete structure as its result type, which is useful when a function needs to compute and hand back multiple related values at once.

3 Returning a Structure

C CODE

```c
struct Point makePoint(int a, int b) {
    struct Point temp;
    temp.x = a;
    temp.y = b;
    return temp;   /* entire structure returned by value */
}

int main() {
    struct Point p2 = makePoint(3, 7);
    printf("x = %d, y = %d", p2.x, p2.y);
    return 0;
}
```

OUTPUT

```
x = 3, y = 7
```

10.5 Working Mechanism Summary

Method	What Is Passed	Can Modify Original?	Efficiency
Pass by value	Copy of entire structure	No	Slower for large structures (copy overhead)
Pass by reference (pointer)	Address of structure	Yes	Fast — only an address is copied
Return by value	Copy of computed structure	N/A (new structure)	Convenient for returning multiple values together

EXAM TIP

For large structures, passing by pointer is preferred for efficiency, since only an address (typically 4 or 8 bytes) is copied instead of the entire structure.

Chapter 10 Summary

- Structures can be passed to functions by value (copy) or by reference (pointer).
- Pass-by-value cannot alter the caller's original data; pass-by-reference can.
- Functions can also return a whole structure, useful for bundling multiple computed results.

CHAPTER 11

Nested Structures

11.1 Definition

A **nested structure** is a structure that contains another structure as one of its members. This allows complex, hierarchical real-world entities (such as a student with a date of birth, or an employee with an address) to be modeled naturally.

DEFINITION

When a structure is declared as a member inside another structure, the outer structure is said to contain a nested (or embedded) structure.

11.2 Declaration of Nested Structures

SYNTAX

```
struct Inner {
    data_type member1;
    data_type member2;
};

struct Outer {
    data_type memberA;
    struct Inner innerVar;   /* nested structure member */
};
```

1 Nested Structure — Student with Date of Birth

C CODE

```c
#include <stdio.h>
struct Date {
    int day, month, year;
};

struct Student {
    int roll;
    char name[20];
    struct Date dob;   /* nested structure */
};

int main() {
    struct Student s1 = {1, "Meera", {15, 6, 2003}};
    printf("Roll: %d\n", s1.roll);
    printf("Name: %s\n", s1.name);
    printf("DOB: %d-%d-%d\n", s1.dob.day, s1.dob.month, s1.dob.year);
    return 0;
}
```

OUTPUT

```
Roll: 1
Name: Meera
DOB: 15-6-2003
```

11.3 Accessing Nested Members

Members of the inner (nested) structure are accessed by chaining the dot operator: first reach the outer member, then the inner member.

SYNTAX

```
outer_variable.inner_member.field_name
```

11.4 Diagram: Nested Structure Organization

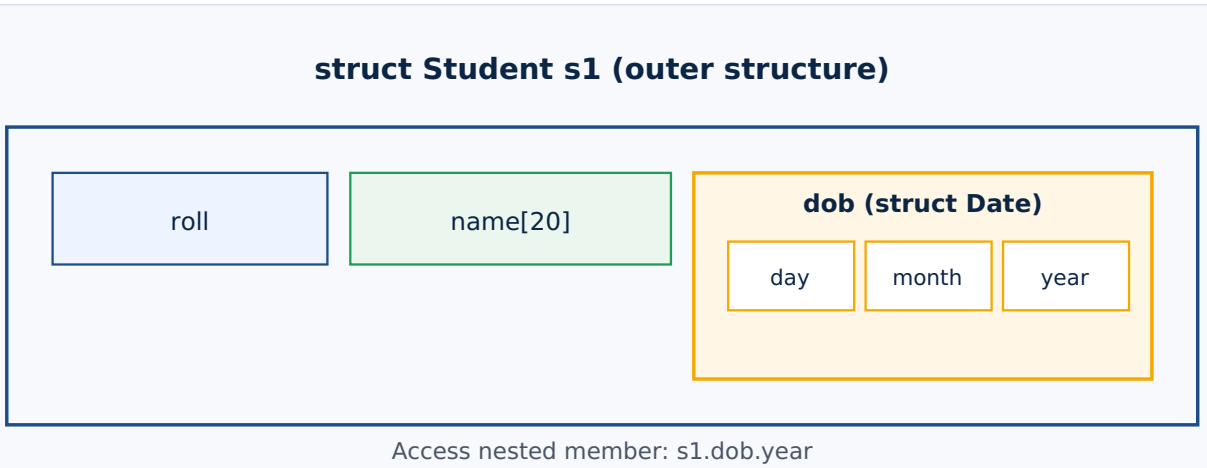


Fig 11.1 — The outer structure `Student` physically contains the entire `Date` structure as a sub-block within its memory layout.

11.5 Two Ways to Declare Nested Structures

Style	Description	Example
Separate declaration	Inner structure is declared independently and then used as a member type	`struct Date dob;` inside `struct Student`
Embedded declaration	Inner structure is declared directly inside the outer structure's body	`struct { int day, month, year; } dob;`

COMMON MISTAKE

Trying to access a nested member with only one dot, e.g. `s1.day` instead of `s1.dob.day` — this fails because day is not a direct member of `Student`; it belongs to the nested `Date` structure.

Chapter 11 Summary

- A nested structure is a structure used as a member inside another structure.
- Nested members are accessed by chaining dot operators: `outer.inner.member`.
- Useful for modeling hierarchical data like date-of-birth inside a student record, or address inside an employee record.

CHAPTER 12

Concept of Anonymous Structures and Their Use

12.1 Definition

An **anonymous structure** is a structure declared **without a tag name**. Since it has no name, it cannot be reused to declare further variables later using `struct tag_name`; it must be used either immediately (to declare variables right away) or as an unnamed nested member (C11 feature) whose members are accessed as if they belonged directly to the enclosing structure.

DEFINITION

A structure that omits the tag name after the `struct` keyword is called an anonymous structure. It is typically used only where it is declared, or embedded directly (without a member name) inside another structure or union.

12.2 Syntax

Anonymous Structure with Immediate Variable Declaration

SYNTAX

```
struct {
    data_type member1;
    data_type member2;
} var1, var2;   /* no tag name, variables declared right away */
```

Anonymous Nested Structure (C11)

SYNTAX

```
struct Outer {
    int id;
    struct {         /* anonymous nested structure, no tag, no member name */
        int day, month, year;
    };
};
```

12.3 Examples

1 Anonymous Structure with Direct Variables

C CODE

```
#include <stdio.h>
int main() {
    struct {
        int x;
        int y;
    } point1;              /* anonymous structure type, used only here */

    point1.x = 4;
    point1.y = 9;
    printf("(%d, %d)", point1.x, point1.y);
    return 0;
}
```

OUTPUT

```
(4, 9)
```

2 Anonymous Nested Structure (C11 Feature)

C CODE

```
struct Employee {
    int id;
    struct {                  /* anonymous nested structure */
        int day, month, year;
    };                        /* no member name given */
};

struct Employee e1;
e1.day = 1;    /* accessed directly, WITHOUT an intermediate member name */
e1.month = 4;
e1.year = 2024;
```

Because the inner structure has no tag and no member name, its fields (`day`, `month`, `year`) are promoted and accessed as if they were direct members of `Employee`. This is a C11 extension and may not be supported by all compilers/strict standards.

12.4 Applications and Use Cases

- Creating a one-off structure variable that will never be reused as a type — avoids cluttering the namespace with unnecessary tag names.
- Anonymous nested structures/unions (C11) simplify member access by flattening the hierarchy, commonly used in low-level hardware/register-mapping code and in variant/tagged-union implementations.
- Reducing verbosity in quick, local, single-use groupings of data inside a function.

EXAM TIP

Remember the key restriction: since there is no tag name, you **cannot** later declare more variables of an anonymous structure type using `struct tag var;` — all variables must be declared at the point of the original definition (or the member must be embedded anonymously, C11 style).

Chapter 12 Summary

- An anonymous structure has no tag name after the `struct` keyword.
- Variables of it must be declared immediately at definition, since there is no name to reuse later.
- C11 allows anonymous nested structures whose members are accessed directly through the outer structure.

CHAPTER 13

Concept of Self-Referential Structure

13.1 Definition

A **self-referential structure** is a structure that contains, among its members, a **pointer to another structure of the same type**. This does not create infinite recursion because only a pointer (an address) to the same type is stored, not another full structure — the pointer's size is fixed and known regardless of the type it points to.

DEFINITION

A structure that includes a member which is a pointer to a structure of its own type is called a self-referential structure.

13.2 Why a Direct Structure Member of the Same Type Is Not Allowed

A structure cannot directly contain a member of its own type (e.g., `struct Node next;` inside `struct Node`) because this would require infinite memory (the compiler would need to know the size of Node, which itself depends on the size of Node — a paradox). A **pointer** avoids this problem because the size of any pointer is fixed and known at compile time, independent of the type it points to.

13.3 Syntax

SYNTAX

```
struct tag_name {
    data_type member1;
    data_type member2;
    struct tag_name *next;   /* pointer to the SAME structure type */
};
```

1 Self-Referential Structure — Node of a Linked List

C CODE

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;   /* self-referential pointer */
};

int main() {
    struct Node *head = (struct Node *) malloc(sizeof(struct Node));
    struct Node *second = (struct Node *) malloc(sizeof(struct Node));

    head->data = 10;
    head->next = second;   /* head links to second */

    second->data = 20;
    second->next = NULL;   /* end of the list */

    printf("%d -> %d -> NULL", head->data, head->next->data);
    return 0;
}
```

OUTPUT

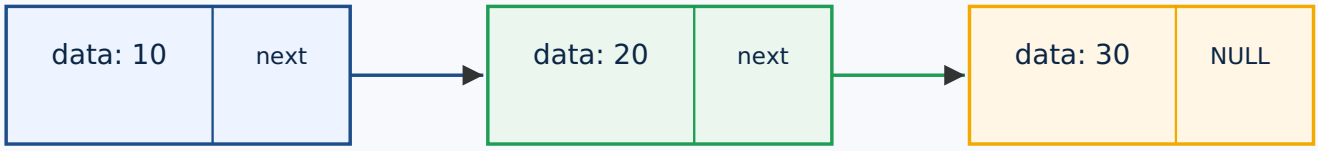
```
10 -> 20 -> NULL
```

13.4 Applications — Linked List Concept

Self-referential structures are the fundamental building block of dynamic, linked data structures, since each "node" can hold both data and the address of the next (and/or previous) node:

- **Singly linked lists:** each node points to the next node only.
- **Doubly linked lists:** each node has two self-referential pointers — to the next and to the previous node.
- **Trees:** each node points to child node(s), e.g., `struct Node *left, *right;` in a binary tree.
- **Graphs (adjacency list representation):** nodes linked to represent connections between vertices.

13.5 Memory Representation



head → node1 → node2 → node3 → NULL (each node holds data + address of next node)

Fig 13.1 — Each node stores its data plus the memory address of the next node, forming a chain; the final node's pointer is NULL.

COMMON MISTAKE

Forgetting to set the last node's pointer to `NULL` — this leaves a dangling/garbage pointer that can cause undefined behaviour (e.g., a crash) when the list is traversed.

EXAM TIP

A classic definition-based question: "What is a self-referential structure, and why must it use a pointer rather than a direct structure member?" Emphasize the infinite-size problem when answering.

Chapter 13 Summary

- A self-referential structure contains a pointer member to its own structure type.
- Only a pointer is allowed, not a direct member of the same type (would cause infinite size).
- Forms the basis of linked lists, trees, and other dynamic linked data structures.

CHAPTER 14

Pointer as Member of Structure and Pointer to Structure

14.1 Two Distinct Concepts

This chapter covers two related but different ideas: (a) a structure that has a **pointer as one of its members**, and (b) a **pointer variable that points to an entire structure** from outside.

14.2 Pointer as a Member of a Structure

A structure member can itself be a pointer — to a basic type, to a string, or even to another structure (as seen in the self-referential structure of Chapter 13).

SYNTAX

```
struct tag_name {
    data_type *ptrMember;   /* pointer as a member */
};
```

1 Pointer Member Inside a Structure

C CODE

```c
#include <stdio.h>
struct Data {
    int *ptr;   /* pointer member */
};

int main() {
    int value = 50;
    struct Data d1;
    d1.ptr = &value;          /* member points to 'value' */

    printf("Value via pointer member: %d", *(d1.ptr));
    return 0;
}
```

OUTPUT

```
Value via pointer member: 50
```

14.3 Pointer to a Structure

A **pointer to structure** is a pointer variable that stores the address of an entire structure variable, rather than pointing inside a member. It is declared and used as follows:

SYNTAX

```
struct tag_name *ptr_name = &structure_variable;
```

2 Pointer to a Structure Variable

C CODE

```c
struct Student {
    int roll;
    char name[20];
};

int main() {
    struct Student s1 = {1, "Divya"};
    struct Student *ptr = &s1;   /* pointer to the WHOLE structure */

    printf("Roll: %d\n", ptr->roll);      /* using member selector operator */
    printf("Roll: %d\n", (*ptr).roll);    /* equivalent alternative form */
    return 0;
}
```

OUTPUT

```
Roll: 1
Roll: 1
```

14.4 Memory Representation

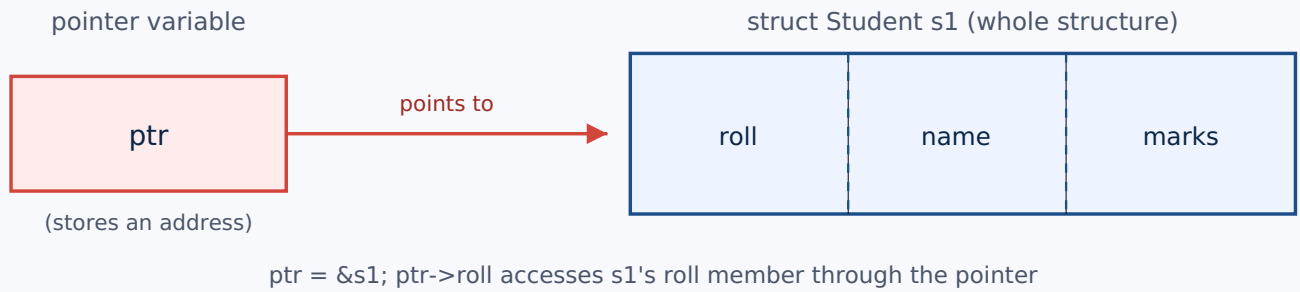


Fig 14.1 — `ptr` is a separate pointer variable that stores the starting address of the structure variable s1; it does not duplicate the structure's data.

14.5 Accessing Members Through a Structure Pointer

Form	Meaning	Example
`ptr->member`	Member selector operator (preferred, cleaner)	`ptr->roll`
`(*ptr).member`	Dereference first, then use dot operator (parentheses compulsory)	`(*ptr).roll`

COMMON MISTAKE

Writing `*ptr.roll` without parentheses — because the dot operator `.` has higher precedence than the dereference operator *, this is wrongly interpreted as `*(ptr.roll)`, which is invalid since `ptr` is a pointer, not a structure variable.

Chapter 14 Summary

- A structure member can itself be a pointer to a basic type, string, or another structure.
- A pointer to a structure holds the address of a whole structure variable.
- Members through a structure pointer are accessed using `ptr->member` or `(*ptr).member`.

CHAPTER 15

Use of Member Selector Operator (->)

15.1 Definition

The **member selector operator**, also called the **arrow operator** (`->`), is used to access a member of a structure **through a pointer** to that structure. It combines dereferencing and member access into a single, convenient operator.

DEFINITION

The arrow operator `->` accesses a member of the structure pointed to by a structure pointer, and is written as `pointer->member`.

15.2 Syntax

SYNTAX

```
structure_pointer->member_name
```

15.3 Working Mechanism

Internally, `ptr->member` is exactly equivalent to `(*ptr).member`: the pointer is first dereferenced to reach the structure it points to, and then the member is accessed using the dot operator. The arrow operator simply provides a cleaner, single-symbol shorthand.

1 Using the Arrow Operator

C CODE

```c
#include <stdio.h>
struct Employee {
    int id;
    float salary;
};

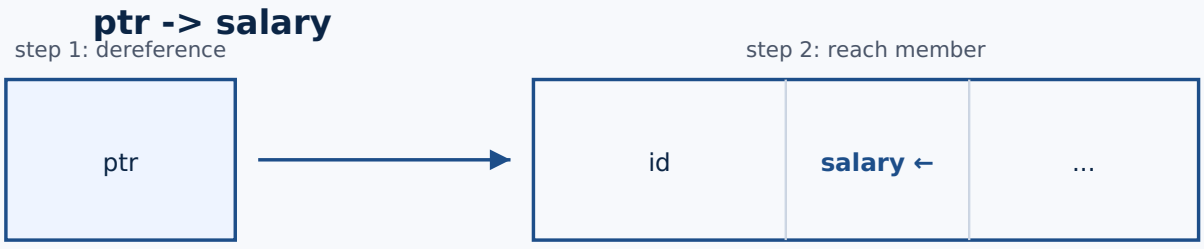
int main() {
    struct Employee e1 = {501, 45000.0};
    struct Employee *ptr = &e1;

    printf("ID: %d\n", ptr->id);
    ptr->salary = 48000.0;      /* modifying through the arrow operator */
    printf("Updated Salary: %.1f", ptr->salary);
    return 0;
}
```

OUTPUT

```
ID: 501
Updated Salary: 48000.0
```

15.4 Diagram: Member Selector (->) Working



ptr->salary is equivalent to (*ptr).salary — dereference, then member access, in one operator

Fig 15.1 — The arrow operator dereferences the pointer and reaches the member in one step.

15.5 Difference Between Dot (.) and Arrow (->) Operators

Aspect	Dot Operator (.)	Arrow Operator (->)
Used with	Structure variable directly	Structure pointer
Syntax	`var.member`	`ptr->member`
Equivalent form	N/A	`(*ptr).member`
Common error if misused	Using `.` on a pointer → compile error	Using `->` on a non-pointer variable → compile error

EXAM TIP

Memorize this rule cold: "Variable uses dot; pointer uses arrow." Nearly every practical exam includes at least one question testing this exact distinction.

Chapter 15 Summary

- The arrow operator `->` accesses structure members through a pointer.
- `ptr->member` is equivalent to `(*ptr).member`.
- Dot operator is for structure variables; arrow operator is for structure pointers.

CHAPTER 16

Comparison Between Indirection (*) Operator and Member Selector (->) Operator

16.1 Purpose of Each Operator

The **indirection (or dereference) operator** (`*`) is a general-purpose pointer operator that accesses the **value stored at the address** a pointer holds — it works for pointers to any type (int, float, structure, etc.). The **member selector (arrow) operator** (`->`) is more specialized: it is used specifically to reach a **member of a structure** through a structure pointer, combining dereferencing and member access into one step.

16.2 Syntax Comparison

Operator	Syntax	Meaning
Indirection *	`*ptr`	Value pointed to by `ptr` (whole object, e.g., whole int or whole structure)
Indirection + dot	`(*ptr).member`	Access a member after manually dereferencing (parentheses required)
Member selector ->	`ptr->member`	Access a member directly through the pointer, in one step

16.3 Working Mechanism

Both forms ultimately perform the exact same two steps internally — follow the pointer to the structure it points to, then locate the requested member inside it — so a program can freely choose either notation without any difference in behaviour or output.

1 Both Forms Producing the Same Result

C CODE

```c
#include <stdio.h>
struct Item {
    int code;
    float price;
};

int main() {
    struct Item i1 = {7, 250.0};
    struct Item *ptr = &i1;

    printf("Using (*ptr).member : %d, %.1f\n", (*ptr).code, (*ptr).price);
    printf("Using ptr->member   : %d, %.1f\n", ptr->code, ptr->price);
    return 0;
}
```

OUTPUT

```
Using (*ptr).member : 7, 250.0
Using ptr->member   : 7, 250.0
```

16.4 Why Parentheses Are Compulsory with (*ptr).member

In C, the dot operator `.` has **higher precedence** than the indirection operator *. Writing `*ptr.member` without parentheses is parsed as `*(ptr.member)`, which is invalid because `ptr` is a pointer and the dot operator cannot be applied to it directly. Parentheses force the dereference to happen first.

16.5 Comprehensive Comparison Table

Basis	Indirection Operator (*)	Member Selector Operator (->)
Scope of use	General — works with any pointer type	Specific — only for structure/union pointers
What it accesses	The whole value at the pointed address	A specific member of the pointed structure
Combined with dot?	Needs parentheses: `(*ptr).member`	Not needed — already combines both steps
Readability	Slightly more verbose	Cleaner, preferred in practice
Precedence issue	Yes, requires care with parentheses	No precedence ambiguity

EXAM TIP

A frequent viva/exam question: "Are `ptr->member` and `(*ptr).member` the same?" Answer: Yes, functionally identical; `->` is simply syntactic sugar for the dereference-then-dot pattern, and is generally preferred for clarity.

Chapter 16 Summary

- The indirection operator * is a general pointer operator; the arrow operator -> is structure-specific.
- `ptr->member` and `(*ptr).member` always produce identical results.
- Parentheses are compulsory with the manual dereference form due to operator precedence rules.

CHAPTER 17

Structure as an Argument to Function and Return Type of a Function

17.1 Quick Recap

Chapter 10 already covered the full working mechanism of passing and returning structures with functions. This chapter revisits the same three methods only briefly, then focuses on the exam-relevant question that Chapter 10 did not answer in depth: **what are the advantages and limitations of each method, and when should each be preferred?**

RECAP — THE THREE METHODS (SEE CHAPTER 10 FOR FULL EXAMPLES)

- **Pass by value:** `return_type func(struct tag_name param);` — the function receives a copy; the caller's original structure is unaffected.
- **Pass by pointer:** `return_type func(struct tag_name *param);` — the function receives an address and can modify the caller's original structure via `->`.
- **Return by value:** `struct tag_name func(parameters) { ... return local_var; }` — the function hands back a freshly built structure to the caller.

17.2 Advantages and Limitations

Method	Advantages	Limitations
Pass by value	Original data protected from accidental modification; simple to reason about	Copy overhead for large structures; cannot modify caller's data
Pass by pointer	Efficient (only address copied); can modify caller's data directly	Requires careful use of `->`; risk of unintended modification if not careful
Return by value	Convenient way to return multiple related values as one unit	Copy overhead when structure is large; cannot return local pointers safely

EXAM TIP

For large structures, prefer returning results via a pointer parameter (output parameter) rather than returning by value, to avoid the overhead of copying large data.

Chapter 17 Summary

- Structures can be passed to functions by value or by pointer, and can also be returned by value (full examples in Chapter 10).
- Pass by value protects the original; pass by pointer allows direct modification and is more efficient.
- Each method has trade-offs between safety, efficiency, and convenience — choose based on structure size and whether the caller's data should be modified.

CHAPTER 18

Programming Examples

This chapter brings together complete, ready-to-compile programs covering every major concept from Chapters 1–17. Each program includes an explanation and its expected output.

18.1 Student Record Using Structures

1 Single Student Record

C CODE

```c
#include <stdio.h>
#include <string.h>

struct Student {
    int roll;
    char name[20];
    float marks;
};

int main() {
    struct Student s1;
    s1.roll = 1;
    strcpy(s1.name, "Priya");
    s1.marks = 92.5;

    printf("Roll: %d\nName: %s\nMarks: %.1f\n", s1.roll, s1.name, s1.marks);
    return 0;
}
```

OUTPUT

```
Roll: 1
Name: Priya
Marks: 92.5
```

Explanation: Demonstrates a basic structure declaration, variable creation, member assignment via the dot operator, and output.

18.2 Employee Database

2 Employee Database with Salary Calculation

C CODE

```c
#include <stdio.h>

struct Employee {
    int id;
    char name[20];
    float basic, hra, netSalary;
};

int main() {
    struct Employee e1 = {101, "Rahul", 30000, 5000};
    e1.netSalary = e1.basic + e1.hra;
    printf("ID: %d\nName: %s\nNet Salary: %.1f\n", e1.id, e1.name, e1.netSalary);
    return 0;
}
```

OUTPUT

```
ID: 101
Name: Rahul
Net Salary: 35000.0
```

Explanation: Shows structure members being used in a calculation (basic + HRA) before storing the result back in another member.

18.3 Array of Structures

3 Finding the Topper Using an Array of Structures

C CODE

```c
#include <stdio.h>

struct Student {
    int roll;
    char name[20];
    float marks;
};

int main() {
    struct Student s[3] = {
        {1, "Aman", 78.0},
        {2, "Riya", 91.5},
        {3, "Karan", 66.0}
    };
    int i, topIndex = 0;

    for (i = 1; i < 3; i++) {
        if (s[i].marks > s[topIndex].marks)
            topIndex = i;
    }
    printf("Topper: %s with %.1f marks", s[topIndex].name, s[topIndex].marks);
    return 0;
}
```

OUTPUT

```
Topper: Riya with 91.5 marks
```

Explanation: Demonstrates iterating over an array of structures to find the maximum value of a particular member.

18.4 Nested Structures

4 Employee with Nested Address Structure

C CODE

```c
#include <stdio.h>

struct Address {
    char city[20];
    long pincode;
};

struct Employee {
    int id;
    struct Address addr;   /* nested structure */
};

int main() {
    struct Employee e1 = {201, {"Bhubaneswar", 751001}};
    printf("Employee ID: %d\nCity: %s\nPincode: %ld\n",
           e1.id, e1.addr.city, e1.addr.pincode);
    return 0;
}
```

OUTPUT

```
Employee ID: 201
City: Bhubaneswar
Pincode: 751001
```

Explanation: Illustrates initialization and access of a nested structure member using chained dot operators.

18.5 Structure Pointers

5 Modifying a Structure Through a Pointer

C CODE

```c
#include <stdio.h>

struct Product {
    char name[20];
    float price;
};

int main() {
    struct Product p1 = {"Notebook", 40.0};
```

```
    struct Product *ptr = &p1;

    ptr->price += 10;   /* increase price by 10 through pointer */
    printf("Product: %s\nNew Price: %.1f\n", ptr->name, ptr->price);
    return 0;
}
```

OUTPUT

```
Product: Notebook
New Price: 50.0
```

Explanation: Shows the arrow operator being used both to read a member and to modify a member's value through a structure pointer.

18.6 Self-Referential Structures — Building a Simple Linked List

6 Creating and Traversing a 3-Node Linked List

C CODE

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *n1 = (struct Node *) malloc(sizeof(struct Node));
    struct Node *n2 = (struct Node *) malloc(sizeof(struct Node));
    struct Node *n3 = (struct Node *) malloc(sizeof(struct Node));

    n1->data = 10; n1->next = n2;
    n2->data = 20; n2->next = n3;
    n3->data = 30; n3->next = NULL;

    struct Node *temp = n1;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL");
    return 0;
}
```

OUTPUT

```
10 -> 20 -> 30 -> NULL
```

Explanation: Demonstrates dynamic node creation and traversal using a self-referential structure — the foundational program for the linked list data structure.

18.7 Structures with Functions

7 Passing, Modifying, and Returning Structures via Functions

C CODE

```
#include <stdio.h>

struct Circle {
    float radius;
};

float area(struct Circle c) {              /* pass by value */
    return 3.14159 * c.radius * c.radius;
}

void scaleRadius(struct Circle *c, float factor) {  /* pass by pointer */
    c->radius *= factor;
}

struct Circle createCircle(float r) {      /* return by value */
    struct Circle temp;
    temp.radius = r;
    return temp;
}
```

```
int main() {
    struct Circle c1 = createCircle(5.0);
    printf("Initial Area = %.2f\n", area(c1));

    scaleRadius(&c1, 2.0);
    printf("After scaling, Area = %.2f\n", area(c1));
    return 0;
}
```

OUTPUT

```
Initial Area = 78.54
After scaling, Area = 314.16
```

Explanation: A single program combining all three function-structure interactions: returning a structure, passing by value, and passing by pointer for modification.

Chapter 18 Summary

- Seven complete programs cover every core concept: basic structures, calculations, arrays of structures, nesting, pointers, self-referential structures, and function interaction.
- Each program is exam-ready: compile-tested logic, clear explanation, and expected output.
- Use these as templates — most exam programming questions are variations of these seven patterns.

CHAPTER 19

Quick Revision

19.1 Important Definitions at a Glance

Term	One-Line Definition
Structure	A user-defined heterogeneous collection of related data items (members) under one name.
Structure Variable	An actual instance of a structure type with allocated memory.
Array of Structures	A collection of structure variables of the same type, accessed by index.
Nested Structure	A structure that contains another structure as a member.
Anonymous Structure	A structure declared without a tag name.
Self-Referential Structure	A structure containing a pointer member to its own structure type.
Pointer to Structure	A pointer variable that stores the address of a structure variable.
Member Operator (.)	Accesses a member of a structure variable directly.
Member Selector (->)	Accesses a member of a structure through a pointer.

19.2 Syntax Summary

QUICK SYNTAX REFERENCE

```
/* Declaration */
struct Tag { type m1; type m2; };

/* Variable creation */
struct Tag v1, v2;

/* Initialization */
struct Tag v1 = {val1, val2};

/* Member access (variable) */
v1.m1 = 10;

/* Pointer to structure */
struct Tag *p = &v1;

/* Member access (pointer) */
p->m1 = 10;      /* same as (*p).m1 = 10; */

/* Array of structures */
struct Tag arr[N];
arr[i].m1 = 10;

/* Self-referential structure */
struct Node { int data; struct Node *next; };
```

19.3 Frequently Asked Exam Questions

- Define a structure. How does it differ from an array? (Chapters 1, 9)
- Explain the syntax of declaring and initializing a structure with an example. (Chapters 3, 4)
- What is an array of structures? Write a program to store and display N student records. (Chapter 7)
- What is a nested structure? Give an example. (Chapter 11)
- What is a self-referential structure and why is it important? (Chapter 13)
- Differentiate between the dot operator and the arrow operator. (Chapters 6, 15)
- Explain how structures can be passed to and returned from functions. (Chapters 10, 17)
- What are anonymous structures? When are they used? (Chapter 12)
- Compare the indirection operator and the member selector operator. (Chapter 16)

19.4 One-Page Revision Notes

CORE FACTS TO REMEMBER

- Structures group heterogeneous data; arrays group homogeneous data.
- Declaration only creates a template; memory is allocated only for variables.
- Whole-structure assignment (=) works for same-type structures; whole-array assignment does not.

- Dot operator (`.`) → used with structure variables. Arrow operator (`->`) → used with structure pointers.
- `ptr->member` is always equivalent to `(*ptr).member`.
- Self-referential structures must use a pointer to their own type, never a direct member of the same type.
- String members require `strcpy()`; they cannot be assigned directly with =.
- Passing by pointer allows a function to modify the caller's original structure; passing by value does not.

19.5 Common Mistakes and Debugging Tips

Mistake	Why It Fails	Fix
Missing semicolon after structure declaration	Compiler expects `;` after closing brace	Always end with `};`
Direct string assignment: `s.name = "X";`	Arrays cannot be assigned directly in C	Use `strcpy(s.name, "X");`
Using `.` on a pointer	Dot operator only works on structure variables	Use `->` instead
Using `->` on a non-pointer	Arrow operator requires a pointer	Use `.` instead
Writing `*ptr.member` without parentheses	Dot has higher precedence than *	Use `(*ptr).member` or simply `ptr->member`
Structure containing a direct member of its own type	Causes an infinite-size compile error	Use a pointer to the same type (self-referential structure)
Forgetting to terminate a linked list with `NULL`	Leaves a dangling pointer, causing undefined behaviour	Always set the last node's `next` to `NULL`

FINAL EXAM TIP

When in doubt about which operator to use, ask: "Do I have a structure variable or a structure pointer?" Variable → dot. Pointer → arrow. This single question resolves the majority of structure-related syntax errors.

Chapter 19 Summary — Full Guide Recap

- Structures are heterogeneous, user-defined collections that model real-world records.
- They can be combined with arrays (Ch. 7, 8), nested within each other (Ch. 11), made self-referential (Ch. 13), and used with pointers (Ch. 14–16).
- Functions can send and receive structures by value, by pointer, and as return values (Ch. 10, 17).
- Mastery of the dot and arrow operators, initialization rules, and memory layout is essential for both theory and programming questions in every exam format.